ONBRD-05: Deploying OneAgent

> **Series:** ONBRD — Dynatrace Onboarding | **Notebook:** 5 of 10 | **Created:** December 2025 | **Last Updated:** 11/13/2026

Getting Data Into Dynatrace

OneAgent is the foundation of Dynatrace monitoring. This notebook covers deployment strategies, installation methods, and verification steps to ensure your infrastructure is reporting data.

> **On Dynatrace Managed?** This notebook is written for SaaS. How OneAgent reaches a Managed cluster, host groups and network zones to decide before rollout, and update policy on Managed are covered in DTM-05: OneAgent Deployment on Managed.

Table of Contents

1. What is OneAgent?
2. Deployment Strategy
3. Generating a Deployment Token
4. Installation Methods
5. Kubernetes Deployment
6. Verifying Deployment
7. Troubleshooting
8. Next Steps

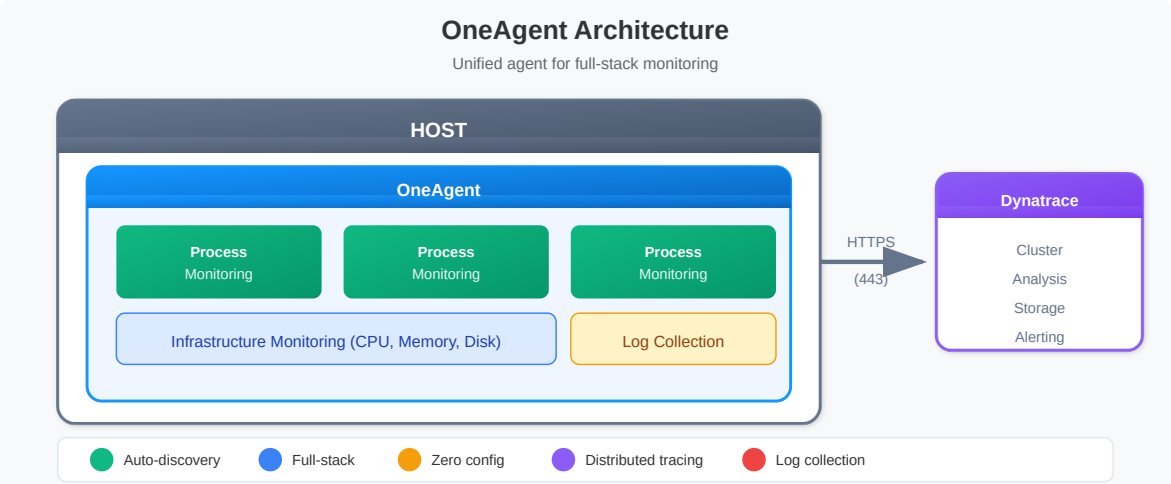
Prerequisites

- Admin access to Dynatrace environment
- A token that can download the OneAgent installer — a platform token with `fleet-management:oneagents:download`, or, where classic access tokens still exist, an access token with `InstallerDownload` (see section 3)
- Root/admin access to target hosts
- Network access from hosts to Dynatrace (port 443) or ActiveGate (ONBRD-03)
- Cloud integrations configured (ONBRD-04) for full context

1. What is OneAgent?

OneAgent is Dynatrace's unified monitoring agent that:

Capability	Description
Auto-discovery	Automatically detects hosts, processes, services
Full-stack	Infrastructure, application, and user experience
Zero configuration	Works immediately after installation
Distributed tracing	End-to-end transaction visibility
Log collection	Collects and forwards log data



2. Deployment Strategy

Phased Rollout (Recommended)

Don't deploy everywhere at once. Use a phased approach:



Phase	Scope	Purpose
Pilot	2-5 non-critical hosts	Validate installation, network, discovery
Expand	One application tier	Test service detection, tracing
Production	Full environment	Complete coverage

Deployment Methods by Environment

Environment	Recommended Method
Bare metal/VMs	Direct install or package manager
Kubernetes	Dynatrace Operator
OpenShift	Dynatrace Operator
AWS ECS	Task definition sidecar
Azure	VM Extension or AKS Operator

Environment	Recommended Method
GCP	Direct install or GKE Operator

Network Requirements

OneAgent needs outbound HTTPS (443) to:
- `{tenant-id}.live.dynatrace.com` (SaaS) — the host the Deployment API serves the installer from
- every endpoint `oneagentctl --get-server` reports once the agent is installed — the parameter is used *"to display the endpoints that OneAgent is to send the data to. These can be Dynatrace Server, Dynatrace Managed Cluster or ActiveGate."*

The `{tenant-id}.apps.dynatrace.com` domain is where the platform UI and apps live; the classic `/api/v1/deployment/...` paths are not served there.

If direct access isn't possible, OneAgents connect through **ActiveGate** (see ONBRD-03).

> **Sources:** Deployment API - Download latest OneAgent (DT docs) — *"GET SaaS https://{your-environment-id}.live.dynatrace.com/api/v1/deployment/installer/agent/{osType}/{installerType}/latest"*, OneAgent configuration via CLI (DT docs).

3. Generating a Deployment Token

The installer download accepts two kinds of token. Which one you have depends on the state of your environment: on a latest environment, *"Platform tokens only. Classic access tokens don't exist, and v2/apiTokens isn't available."*, while environments migrating to Latest Dynatrace have both.

Environment	Token	Required scope	Request header
Latest Dynatrace (platform tokens only)	Platform token (or OAuth client)	`fleet-management:oneagents:download`	`Authorization: Bearer <token>`

Environment	Token	Required scope	Request header
Classic or hybrid (classic access tokens still exist)	Access token — **Access Tokens** app → **Generate new token**	`InstallerDownload` (*PaaS integration - Installer download*)	`Authorization: Api-Token <token>`

Create new automation with a platform token where your environment offers one; keep a classic `InstallerDownload` token only where scripts still depend on it.

Token Naming Convention

Use descriptive names:
- `prod-oneagent-deployment`
- `dev-installer-token`
- `k8s-operator-token`

> **Important:** Copy the token immediately after creation. It won't be shown again.
>
> **Sources:** Deployment API - Download latest OneAgent (DT docs) — *"Platform Token / OAuth: Required scope: fleet-management:oneagents:download"*, Upgrade from access tokens classic (DT docs) — *"Replace Authorization: Api-Token with Authorization: Bearer in every migrated integration."*

4. Installation Methods

Linux (Direct Download)

The Deployment API is served from `{tenant-id}.live.dynatrace.com`. Pass the token in the `Authorization` header — Dynatrace recommends *"the header approach, as URLs (along with tokens passed within them) might be logged in various locations."*

```
# Download the installer (platform token; for a classic access token use
# --header="Authorization: Api-Token {token}" instead)
wget -O Dynatrace-OneAgent.sh \
  --header="Authorization: Bearer {platform-token}" \
  "https://{tenant-
id}.live.dynatrace.com/api/v1/deployment/installer/agent/unix/default/latest?
arch=x86&amp;flavor=default"

# Run the installer
sudo /bin/sh Dynatrace-OneAgent.sh
```

Linux (One-liner)

```
wget -O Dynatrace-OneAgent.sh --header="Authorization: Bearer {platform-
token}" "https://{tenant-
id}.live.dynatrace.com/api/v1/deployment/installer/agent/unix/default/latest?
arch=x86&amp;flavor=default" &amp;&amp; sudo /bin/sh Dynatrace-OneAgent.sh
```

Windows (PowerShell)

```
# Download the installer (classic access token: "Api-Token {token}" instead of
"Bearer ...")
Invoke-WebRequest -Uri "https://{tenant-
id}.live.dynatrace.com/api/v1/deployment/installer/agent/windows/default/lates
t" `
  -Headers @{ Authorization = "Bearer {platform-token}" } -OutFile Dynatrace-
OneAgent.exe

# Run the installer
.\Dynatrace-OneAgent.exe
```

> **Sources:** Deployment API - Download latest OneAgent (DT docs), Dynatrace API - Authentication (DT docs), Platform tokens (DT docs) — *"Authorization: Bearer "*
>
> **OneAgent 1.337 Windows network insight (Npcap):** *"OneAgent on Windows now requires Npcap to provide network metrics generated by Network Agent. Winpcap is no longer supported."* Plan an Npcap install / redistribution into your Windows golden image when rolling out OneAgent on hosts that need network insight. Source: What's new in OneAgent 1.337 (DT docs).

Setting Primary Tags (OneAgent 1.333+)

Set `dt.security_context`, environment, team, cost-center, and other primary tags with `--set-host-tag` — on the installer command line at install time, or with `oneagentctl` on a host that is already running. Primary tags are *"identified by the reserved primary_tags.prefix"* (Primary Grail fields and tags enrichment through OneAgent (DT docs)), so write it out in every key. Primary tags emit at the source on every signal (metrics, spans, logs, events) and feed directly into OpenPipeline routing, bucket assignment, and IAM scoping — more efficient than view-time auto-tagging.

```
# At install time (preferred): pass primary tags and the host group to the installer
# The primary_tags. prefix must be written explicitly - without it you set a classic host tag
sudo /bin/sh Dynatrace-OneAgent.sh \
  --set-host-tag="primary_tags.environment=prod" \
  --set-host-tag="primary_tags.team=payments" \
  --set-host-tag="dt.security_context=team-payments" \
  --set-host-group=prod-app-payments

# On an existing host: oneagentctl set parameters apply only after a OneAgent restart,
# so chain them into one command with --restart-service
sudo /opt/dynatrace/oneagent/agent/tools/oneagentctl \
  --set-host-tag="primary_tags.environment=prod" \
  --set-host-tag="primary_tags.team=payments" \
  --set-host-tag="dt.security_context=team-payments" \
  --restart-service
```

> **Sources:** Primary Grail fields and tags enrichment through OneAgent (DT docs) — *"You can set primary Grail fields and tags at installation time by passing --set-host-tag parameters directly to the OneAgent installer."*, OneAgent configuration via CLI (DT docs) — *"When you use the set parameters, you need to restart OneAgent service to apply changes."*

Decide your primary-tag and host-group taxonomy *before* the first install — retrofitting breaks history-based thresholds, IAM scoping, and any automation keyed on those values. See **FAQ-01 (host group naming strategy)** and **FAQ-02 (tagging sources, standards, strategy)** for the canonical guidance.

Configuration Management

For orchestrated rollout, Dynatrace provides an Ansible collection — *"Currently Dynatrace provides you with an Ansible collection."* See Install OneAgent using Ansible (DT docs).

The Dynatrace Puppet module and Chef cookbook repositories are archived — Puppet: *"Official support for this project has been discontinued."* Treat them as unmaintained code; with Puppet, Chef, or any other tool, wrap the installer download and command line shown above.

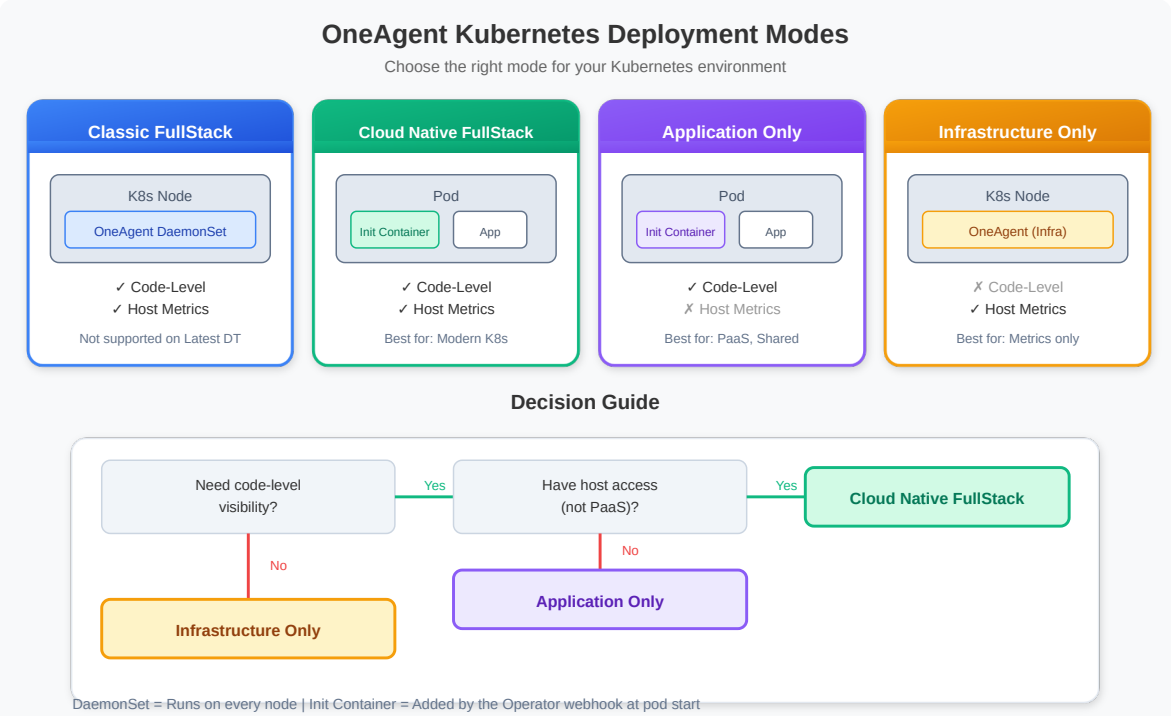
> **Sources:** OneAgent deployment orchestration (DT docs), Dynatrace-OneAgent-Puppet (Dynatrace GitHub), Dynatrace-OneAgent-Chef (Dynatrace GitHub) — *"This repository has been archived and is no longer maintained."*

5. Kubernetes Deployment

For Kubernetes environments, use the Dynatrace Operator—the recommended approach for deploying and managing OneAgent in containerized environments.

OneAgent Deployment Modes

The Dynatrace Operator supports multiple deployment modes. Choose based on your requirements:



Mode	Deployment	Code-Level	Host Metrics	Best For
Cloud Native FullStack	Init container; code modules via image volume (Operator 1.11.0+), CSI driver or ephemeral volume	Yes	Yes	Modern K8s (recommended)
Application Only	Init container; same code-module delivery options as Cloud Native FullStack	Yes	No	PaaS, shared nodes, OpenShift
Host Monitoring	DaemonSet	No	Yes	When code-level monitoring not needed

Both code-level modes inject the same way: the Operator's webhook mutates each new pod in a monitored namespace and adds an init container that delivers the OneAgent code modules. Nothing runs alongside the application as a sidecar. Application Only simply has no OneAgent on the node, so there are no host metrics.

> **Sources:** Dynatrace Operator components (DT docs) — the webhook *"Attaches Init container to download (in case no CSI driver is used) and configure the code modules on Pod startup."*

> **Note:** Classic Full-Stack is **not supported** in Latest Dynatrace environments or with a platform token: *"Classic Full-Stack mode is not supported when using a platform token as well as in Latest Dynatrace environments."* Use Cloud Native Full-Stack. (DynaKube parameters (DT docs))

Deployment Method: Helm vs Manifests

Method	Pros	Cons	Best For
Helm Charts	Version management, easy upgrades, values override	Requires Helm	Production, GitOps
Manifest Files	Simple, no dependencies	Manual version tracking	Air-gapped, constrained environments

Prerequisites

- `kubectl` access to cluster
- API token with required scopes
- Cluster admin permissions

Tokens for Kubernetes

On **Latest Dynatrace**, the Operator documentation now directs you to two **platform tokens** — an Operator token and a Data Ingest token — on a dedicated service user: *"For each Kubernetes cluster, Dynatrace Operator uses two platform tokens associated with a dedicated service user"* (Platform tokens for Dynatrace Operator (DT docs)). Existing classic access tokens keep working — the Operator 1.10.0 notes say *"No immediate action is required and existing access tokens continue to be accepted."* See Tokens and permissions (DT docs) for the platform-token scopes, and Dynatrace Operator 1.10.0 (DT docs).

Classic access-token scopes (Dynatrace Classic, or until you move to platform tokens)

Create the Operator token in **Access Tokens** from the *Kubernetes: Dynatrace Operator* template, which adds these scopes:

Scope	Purpose
`InstallerDownload`	Download OneAgent
`entities.read`	Read topology — *no longer required from Operator v1.7.0*
`settings.read`	Manage the ActiveGate object for Kubernetes API monitoring (optional from Operator v1.7.0)
`settings.write`	Manage the ActiveGate object for Kubernetes API monitoring (optional from Operator v1.7.0)
`DataExport`	Export data (optional)
`activeGateTokenManagement.create`	Create ActiveGate token (API v2) — needed for any DynaKube that deploys an ActiveGate

> **Sources:** Tokens and permissions Classic (DT docs) — *"Creates an authentication token for your ActiveGate to connect to the Dynatrace Cluster."*; `entities.read`: *"No longer required with Dynatrace Operator version v1.7.0+"*.

Installation Option 1: Helm (Recommended)

```
# Create namespace and secret
kubectl create namespace dynatrace
kubectl -n dynatrace create secret generic dynakube --from-
literal="apiToken=&lt;API_TOKEN&gt;" --from-
literal="dataIngestToken=&lt;DATA_INGEST_TOKEN&gt;"

# Install from the OCI registry, pinned to the version you validated
helm upgrade dynatrace-operator oci://public.ecr.aws/dynatrace/dynatrace-
operator \
  --version 1.10.4 \
  --namespace dynatrace \
  --install --atomic \
  --set installCRD=true
```

> **Install from the OCI registry.** The Operator 1.11.0 release notes direct Helm installs to `oci://public.ecr.aws/dynatrace/dynatrace-operator`: the legacy `dynatrace/helm-charts` repository is archived, and *"If you can't use OCI, point your*

> *Helm repository to dynatrace/dynatrace-operator instead"* (`helm repo add dynatrace https://raw.githubusercontent.com/Dynatrace/dynatrace-operator/main/config/helm/repos/stable`). `1.10.4` (released 10/22/2026; it adds the CSI driver cleanup fix from 1.10.3 and fixes CVE-2026-111943) is the version validated across these notebooks — it is *"released only on GitHub (Helm and manifests)"*, so OperatorHub and marketplace installs take 1.11.1; Operator **1.11.1** (released 10/22/2026) is the newest — validate it on a non-production cluster before moving the pin.
>
> **Sources:** Operator 1.11.0 release notes (DT docs) — *"The Helm repository located in dynatrace/helm-charts is archived and no longer receives updates."*

Installation Option 2: Manifest Files

1. **Navigate to Kubernetes monitoring**
   - Infrastructure → Kubernetes → Add cluster
2. **Follow the wizard**
   - Select your distribution
   - Choose observability options
   - Download dynakube.yaml
3. **Apply to cluster**
   `bash kubectl apply -f https://github.com/Dynatrace/dynatrace-operator/releases/download/v1.10.4/kubernetes.yaml kubectl apply -f dynakube.yaml`

> **Pin a version you have validated.** The URL above is pinned rather than tracking a moving `latest` on purpose — a manifest install is the path you reach for precisely when you need reproducibility (air-gapped, GitOps, change-controlled clusters). `1.10.4` (published 10/22/2026) is the version validated across these notebooks: it adds the CSI driver cleanup fix from 1.10.3 and fixes CVE-2026-111943 (*"We recommend that you upgrade to this version."* — Operator 1.10.4 release notes (DT docs)). **Skip `1.10.0` :** its own release notes advise waiting for `1.10.1`, and GitHub now marks the `v1.10.0` release a prerelease — the machine-readable trace of that advice. The 1.10.2 release notes (DT docs) published on 07/30/2026 and carry four fixes — a Kubernetes workload/namespace **tagging-precedence regression** (from 1.10.2 only the first matching rule for a key applies, a behavior change as well as a fix), `dynatrace-webhook` `CrashLoopBackOff` on **gVisor** runtime-class nodes, injected

> pods hanging on the OneAgent-binary download (timeout raised to 15 minutes), and metadata-enrichment rules that could not be applied now being logged rather than silently ignored. `1.10.1` and `1.10.2` keep running until you move — estates adopt on their own schedule — but lack those two fixes. **Operator 1.11.1** (released 10/22/2026) is the newest release; **1.11.0** (released 10/01/2026) removes the `v1beta4` DynaKube API from the CRD (below) and adds image-volume code-module injection, and its notes set the upgrade floor at *"Minimum operator version required for direct upgrade: 1.6.0"* — validate it before moving the pin. **On OpenShift, read the 1.10.2 Known Issue before pinning either**: the `RuntimeDefault` seccomp profile applied since 1.9.0 can collide with SecurityContextConstraints — see FAQ-13. Whatever you pin, validate it in a non-production cluster before it reaches the rest of the fleet, and check the Dynatrace Operator releases page for the version current when you read this.

4. **Verify deployment**
   `bash kubectl get pods -n dynatrace`

DynaKube Custom Resource Configuration

The DynaKube CR is the primary configuration for the Dynatrace Operator. Here are examples for each deployment mode:

> **Important:** Use `apiVersion: dynatrace.com/v1beta6` for new DynaKubes (`v1beta5` is still served by Operator 1.11.0 but marked deprecated in its CRD from Operator 1.10.0). Operator **1.9.0** removed `v1beta3` from the CRD (*"Applying DynaKube resources using this version will fail"*) and deprecated `v1beta4`; Operator **1.10.0** (July 15, 2026) stopped serving `v1beta4` (its CRD sets `served: false`, while its notes repeat *"The `v1beta4` version is now deprecated"*); and Operator **1.11.0** (released 10/01/2026) removes it from the CRD — *"Applying DynaKube resources that still use v1beta4 will fail."* Check `kubectl get dynakube -A -o jsonpath='{.items[*].apiVersion}'` before upgrading the Operator, and move any `v1beta4` DynaKube to `v1beta6` first. The examples below use `v1beta6` (validated against the Operator 1.11.0 CRD). v1beta6 adds OTLP exporter configuration.
>
> **Sources:** Operator 1.9.0 release notes (DT docs) — *"Additionally, `v1beta4` is now deprecated."*, Operator 1.10.0 release notes (DT docs), Operator 1.11.0 release notes (DT docs) — *"The v1beta4 version has been removed from the DynaKube CRD."* `served` / `deprecated` per version read

> from the DynaKube CRD in each release's `kubernetes.yaml` (Operator releases (Dynatrace GitHub)), 10/02/2026.

Cloud Native FullStack (Recommended for most K8s):

```
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://{tenant-id}.live.dynatrace.com/api

  # Cloud Native FullStack - automatic init container injection
  oneAgent:
    cloudNativeFullStack:
      tolerations:
        - effect: NoSchedule
          key: node-role.kubernetes.io/master
          operator: Exists
      args:
        - --set-host-group=my-k8s-cluster

  # ActiveGate for Kubernetes API monitoring and OneAgent routing
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
```

> Leave out the `dynatrace-api` capability unless you need it: the DynaKube reference marks it *"A custom certificate is required for this capability"* (set `tlsSecretName`). For production sizing and the two-ActiveGate-set recommendation, see ONBRD-03 § 7.

Application Only (for PaaS or shared nodes):

```
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://{tenant-id}.live.dynatrace.com/api

  # Application Only - init-container injection, no host monitoring
  oneAgent:
    applicationMonitoring: {}
```

> `useCSIDriver` is not a `v1beta5` or `v1beta6` field — the DynaKube parameters reference lists it only for the retired `v1beta1` / `v1beta2` APIs. Through Operator 1.10.x, whether code modules come from the CSI driver is decided when the Operator is installed (CSI or *Without CSI driver* variant), and that remains the working path on those versions.
>
> **Operator 1.11.0+ (released 10/01/2026): image volumes.** Dynatrace now recommends image volume-based code-module injection, which *"replaces the CSI driver as the recommended approach"*: each node pulls the code-modules image once and shares it with every instrumented pod, with no CSI DaemonSet. Turn it on per DynaKube with the annotation `feature.dynatrace.com/mount-code-modules-via-image-volume: "true"` (mutually exclusive with `feature.dynatrace.com/node-image-pull`). It needs **Kubernetes 1.35+** and **containerd 2.2+ or CRI-O 1.33+**, and *"Image volume injection is not compatible with the Dynatrace built-in tenant registry."* Try it on one workload first with the pod annotation `oneagent.dynatrace.com/volume-type: "image"`; *"A full migration requires a rolling restart of all injected workloads."* Clusters that do not meet those requirements stay on the CSI driver or ephemeral volumes. K8S-12 § 2 compares the three delivery modes.
>
> **Sources:**
>
> - DynaKube parameters (DT docs) — *"DynaKube API version v1beta2 is no longer available with Dynatrace Operator version 1.7.0"*
> - Application observability setup (DT docs) — *"CSI driver is optional (see step 2). If enabled, it gets deployed as DaemonSet and results in a CSI driver pod on each node."*
> - Operator 1.11.0 release notes (DT docs)
> - Use image volumes for code modules injection (DT docs) — requirements, both annotations, and

> the registry limit
>
> - Migrate to image volumes (DT docs)

Host Monitoring (host metrics only):

```
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://{tenant-id}.live.dynatrace.com/api

  # Host monitoring only - no code-level visibility
  oneAgent:
    hostMonitoring: {}
```

Air-Gapped / Private Registry Deployment

For environments without internet access, host the images in a private registry. Five things go wrong in an air-gapped rollout more often than the DynaKube itself: the mirror copies one architecture, injected pods have no pull credentials, a component nobody listed still points at a public registry, auto-update silently stops, and the Helm chart itself is fetched from the internet.

```
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://{tenant-id}.live.dynatrace.com/api

  # Pull secret for the Operator-managed components in the dynatrace namespace
only.
  # Injected application pods need their own credentials (see "Pull
credentials" below).
  customPullSecret: my-registry-pull-secret

  # Custom image locations for air-gapped environments
  # Point every image at the mirrored copy, pinned to an immutable tag.
  # Setting image / codeModulesImage turns off auto-update for that component.
  oneAgent:
    cloudNativeFullStack:
      image: my-registry.example.com/dynatrace/dynatrace-
oneagent:1.347.49.20260928-064841
      codeModulesImage: my-registry.example.com/dynatrace/dynatrace-
codemodules:1.347.49.20260928-064841

  activeGate:
    capabilities:
      - kubernetes-monitoring
    image: my-registry.example.com/dynatrace/dynatrace-
activegate:1.345.70.20260918-003121
```

Mirror every architecture, with its signatures. Dynatrace publishes multi-arch images, and the mirroring guide says: *"We strongly recommend that you use the Skopeo CLI instead of Docker CLI for copying Dynatrace container images from public to private registries, as the Docker CLI does not provide an easy way to copy multi-arch images and signatures."* A `docker pull` / `tag` / `push` loop copies only the architecture of the machine that ran it, so a mixed-architecture node pool (K8S-02 §3) would find no image on its other nodes. `skopeo copy --all` copies the whole multi-arch index. Signatures are copied only when Skopeo's registries configuration sets `use-sigstore-attachments` to `true`.

Image Mirroring Script:

```
# Mirror required images to a private registry, keeping every architecture.
# Copy from a supported public registry; the guide says "Please do not use the
Dynatrace
# built-in registry for copying images to private registries" (Classic Full-
Stack excepted).
# These repositories publish no :latest tag - pin immutable tags. The tags
below existed on
# 10/02/2026 (operator: 11/03/2026); pick current ones from
gallery.ecr.aws/dynatrace before you mirror.
OPERATOR_TAG=v1.10.4
ONEAGENT_TAG=1.347.49.20260928-064841
CODEMODULES_TAG=1.347.49.20260928-064841
ACTIVEGATE_TAG=1.345.70.20260918-003121

IMAGES=(
  "public.ecr.aws/dynatrace/dynatrace-operator:${OPERATOR_TAG}"
  "public.ecr.aws/dynatrace/dynatrace-oneagent:${ONEAGENT_TAG}"
  "public.ecr.aws/dynatrace/dynatrace-codemodules:${CODEMODULES_TAG}"
  "public.ecr.aws/dynatrace/dynatrace-activegate:${ACTIVEGATE_TAG}"
)

# Optional: verify the signatures first (requires cosign)
cosign verify \
  --certificate-identity-regexp 'https://github\.com/Dynatrace/dynatrace-
operator/\.github/workflows/.+' \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com \
  "public.ecr.aws/dynatrace/dynatrace-operator:${OPERATOR_TAG}"
for img in "${IMAGES[@]:1}"; do
  cosign verify --insecure-ignore-tlog --key
https://ca.dynatrace.com/v1/cosign.pub "$img"
done

# Copies to my-registry.example.com/dynatrace/&lt;image&gt;:&lt;tag&gt; - the
paths the DynaKube above uses
for img in "${IMAGES[@]}"; do
  skopeo copy --all "docker://${img}" "docker://my-
registry.example.com/${img#*/}"
done
```

> Without `codeModulesImage`, Cloud Native Full-Stack and Application Only still fetch code modules from the tenant. The DynaKube reference says of it: *"Avoid mutable tags such as latest and use a digest or immutable tag for reproducible deployments."* Mirror only what your observability option uses: the guide maps each option to the images it needs.

Pull credentials reach the `dynatrace` namespace only. The *Use a private registry* page says *"The `customPullSecret` field in a DynaKube configuration authenticates only the components that Dynatrace Operator manages in the `dynatrace` namespace."* Injected pods run a Dynatrace init container whose image the node pulls, and the Operator does not copy the secret into application namespaces. Provide credentials at one of three levels: on each **node**, as a pull secret in every application **namespace** and its service accounts, or through `imagePullSecrets` on the **pod**. This matters more from Kubernetes 1.35: *"the `KubeletEnsureSecretPulledImages` feature gate is enabled by default and the kubelet verifies pull credentials per pod, even for images already cached on the node. If you don't distribute pull secrets as described above, injected pods fail with `ImagePullBackOff`."*

Every component you deploy needs a mirrored image. OneAgent, code modules and ActiveGate are only the start. Components configured under `spec.templates` take their own `imageRef` — the log module, Extension Execution Controller, SQL extension executor, OpenTelemetry Collector and Node Configuration Collector. The DynaKube parameters (DT docs) reference states, per field, when `imageRef` is required and from which version it applies, and K8S-02 §2 summarizes which components resolve automatically on which Operator and SaaS versions; the log module's, for example, is marked *"imageRef parameter is Required."* Automatic resolution hands back public-registry image URIs, so in a cluster that cannot reach a public registry, point every component you enable at the mirror.

Pinning an image turns off auto-update for that component. The auto-update guide: *"To disable auto-update, set the image field in the DynaKube. Omit the field to keep auto-update enabled."* With a mirror, updating becomes your job: re-mirror every architecture you run, then move the tag. The Operator is not among the components it updates for you either — upgrade it as in K8S-02 §8.

The Helm chart also comes from a public registry. `helm install` / `helm upgrade` with `oci://public.ecr.aws/dynatrace/dynatrace-operator` pulls the chart from the internet, and the *Use a private registry* page lists *"Allow egress traffic to public registry"* as a prerequisite. For a cluster with no internet path, `helm pull` the chart on a connected machine and install from the `.tgz`, `helm push` it to your own OCI registry, or render manifests with `helm template` and apply those. Point the Operator's own image at the mirror with the chart's `imageRef.repository`, `imageRef.tag` and `customPullSecret` values (K8S-02 §3).

> **Sources:**
>
> - Store Dynatrace images in private registries (DT docs) — *"Requires use-sigstore-attachments to be*

> *set to true in Skopeo's container registries configuration."*
>
> - Verify Dynatrace image signatures (DT docs) — *"For other images, the `--insecure-ignore-tlog` flag is required during verification."*
> - Use a private registry (DT docs)
> - Configure auto-update for Dynatrace Operator managed components (DT docs)
> - DynaKube parameters (DT docs)
> - helm pull (Helm docs) — *"Retrieve a package from a package repository, and download it locally"*; Use OCI-based registries (Helm docs)
> - **Derived:** the single-architecture failure combines Dynatrace's multi-arch images with the Docker CLI limitation quoted above; "automatic resolution hands back public-registry image URIs" reads the auto-update guide's description of resolving *"the latest public image URIs"*; the Operator not being self-updated reads its list of managed components, which does not include it.

Kubernetes Metadata and Dynatrace Tags

Dynatrace can leverage Kubernetes labels and annotations for observability:

Feature	Behavior	Limitations
Pod labels	Appear as `[Kubernetes]` context tags on processes	Requires OneAgent code module injection
Annotations	Available as custom metadata at Process level	Requires view RBAC permissions
Namespace labels	Available via Primary Grail tags	Does not enrich K8s metrics or events

Requirements:
- Service accounts need `view` access via rolebinding/clusterrolebinding
- Labels are read via Kubernetes REST API at deployment time
- Not all entities receive automatic tags (namespaces, pods, workloads have limited support)

For more control, configure automatic tagging rules: **Settings → Automatically applied tags**

Multi-Tenant / Namespace Isolation

For large clusters with multiple teams, give each team's DynaKube a namespace selector — with **Application Only**, because Cloud Native Full-Stack also deploys the host OneAgent to every node it selects:

```
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: team-a-dynakube
  namespace: dynatrace
spec:
  apiUrl: https://team-a-tenant.live.dynatrace.com/api

  oneAgent:
    applicationMonitoring:
      namespaceSelector:
        matchLabels:
          dynatrace-tenant: team-a
```

Keep the selectors disjoint, and let only one DynaKube deploy the host OneAgent on any node (Cloud Native Full-Stack or Host Monitoring, or separate them with non-overlapping `nodeSelector`s). *"Dynatrace Operator rejects a DynaKube that conflicts with another one over OneAgent node assignment, namespace injection, or telemetry ingest service names."*

> **Sources:** Dynatrace Operator 1.9.0 release notes (DT docs) — the `v1beta3` CRD removal quoted above; Consolidate Kubernetes monitoring into a single DynaKube (DT docs).

6. Verifying Deployment

After installation, verify OneAgent is reporting data.

```
// Check all hosts with OneAgent that are reporting now (default 2 h window).
// Hosts that stopped reporting drop out of this window - use the state query
// below (from:-7d) to find OFFLINE hosts.
fetch dt.entity.host
| fields entity.name, state
| filter state == "RUNNING"
| sort entity.name
| limit 50

// Smartscape note (dt.entity.* is deprecated but still functional): this
query uses the
// classic-only field state, which has NO Smartscape node equivalent
// (Smartscape expresses liveness via node lifetime, not a state field). Keep
the classic
// query above for state detail. For monitoring mode, do not use
monitoringMode — it is
// empty on Kubernetes and Fargate hosts; read billing events instead (see the
billing query in this section).
// Other fields do map: entity.name -&gt; name.
```

```
// Check hosts by monitoring mode — read from billing, not from
monitoringMode.
// monitoringMode on dt.entity.host is empty for Kubernetes and Fargate hosts,
even when
// they are billed Full-Stack. A host billed for more than one capability (for
example
// Full-Stack and Code Monitoring) is counted in each row, so do not add the
rows up.
fetch dt.system.events, from:-24h
| filter event.kind == "BILLING_USAGE_EVENT"
// SaaS 1.347 moves the host ID to dt.smartscape.host; coalesce reads
whichever field your tenant writes.
| fieldsAdd host = coalesce(toString(dt.smartscape.host), dt.entity.host)
| filter isNotNull(host)
| summarize {hosts = countDistinctExact(host)}, by:{billed_as = event.type}
| sort hosts desc
```

```
// Check hosts by monitoring state - useful for verifying deployment.
// from:-7d matters: without it, only hosts seen in the default 2 h window are
// returned, so a host that went OFFLINE never appears and every row reads
RUNNING.
fetch dt.entity.host, from:-7d
| summarize host_count = count(), by: {state}
| sort host_count desc

// Smartscape note (dt.entity.* is deprecated but still functional): this
query uses the
// classic-only field state, which has NO Smartscape node equivalent
// (Smartscape expresses liveness via node lifetime, not a state field). Keep
the classic
// query above for state detail. For monitoring mode, do not use
monitoringMode — it is
// empty on Kubernetes and Fargate hosts; read billing events instead (see the
billing query in this section).
// Other fields do map: entity.name -&gt; name.
```

```
// Check discovered services
fetch dt.entity.service
| fields entity.name, serviceType
| summarize service_count = count(), by: {serviceType}
| sort service_count desc

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "SERVICE"
//   | fields name, dt.service.sdv1_type
//   | summarize service_count = count(), by: {dt.service.sdv1_type}
//   | sort service_count desc
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities
// than the classic entity store; for a pre-migration discovery inventory keep
the
// classic query above.
// Field maps: serviceType -&gt; dt.service.sdv1_type; entity.name -&gt; name.
// dt.service.sdv1_type is set only when dt.service_detection.version is 1; on
SDv2 services it is null — add dt.service_detection.version to the by: list to
see which applies.
```

```
// Check for processes discovered
fetch dt.entity.process_group
| fields entity.name
| sort entity.name
| limit 50

// Smartscape note (dt.entity.* is deprecated but still functional):
Smartscape models
// individual processes, not process GROUPS — smartscapeNodes "PROCESS" is a
different
// granularity (process instances), so its results are not comparable to a
process-group
// query. Keep the classic dt.entity.process_group query above.
```

Host Verification Commands

Linux:

```
# Check OneAgent status
sudo systemctl status oneagent

# Show the endpoints OneAgent sends data to (* marks the one in use)
sudo /opt/dynatrace/oneagent/agent/tools/oneagentctl --get-server

# Check OneAgent logs (default log directory)
sudo ls -lt /var/log/dynatrace/oneagent/
```

Windows:

```
# Check service status
Get-Service -Name "Dynatrace OneAgent"

# Show the endpoints OneAgent sends data to
&amp;amp; "$env:ProgramFiles\dynatrace\oneagent\agent\tools\oneagentctl.exe" --
get-server
```

> **Sources:** OneAgent configuration via CLI (DT docs) — *"An asterisk (*) indicates the endpoint to which OneAgent currently sends the data."*, Customize OneAgent installation on Linux (DT docs) — *"Log path Default value : /var/log/dynatrace/oneagent"*

What You'll See on Modern Tenants

Once OneAgent is reporting and services appear in the verification queries above, **Enhanced Endpoints for SDv1** (Dynatrace v1.329+) shapes what shows up under each service:

Tenant creation date	Default state	What you do
v1.333 or later	**Always on, not configurable**	Nothing — each service automatically exposes individual endpoints with `dt.service.request.*` metrics
v1.330 – v1.332	On by default (toggleable)	Verify the toggle hasn't been disabled
v1.329 or earlier	Off by default	Enable at *Settings → Process and contextualize → Services → Service detection v1* if you want per-endpoint metrics

Why it matters during onboarding: before Enhanced Endpoints, only manually marked "key requests" emitted individual metrics — everything else collapsed into a single `NON_KEY_REQUESTS` bucket. Now every detected endpoint is named and tracked automatically, which materially changes what's visible on day one.

Services not affected: external services, background activity, queue listeners, key-value stores — these never get per-endpoint metrics regardless of the setting.

> **Sources:** Enhanced endpoints for SDv1 (DT docs).

7. Troubleshooting

Common Issues

Issue	Cause	Solution
Host not appearing	Network blocked	Check firewall rules for 443

Issue	Cause	Solution
Host showing but no data	Agent not running	Restart OneAgent service
Services not discovered	Processes not restarted	Restart monitored applications
Old OneAgent version	Auto-update disabled	Enable auto-update or manual upgrade
Host stopped reporting after a SaaS update	OneAgent 1.241 or earlier — SaaS 1.347 (staged tenant rollout) rejects these connections once it reaches your tenant	Upgrade the agent; inventory with `fetch dt.entity.host, from:-30d | fields entity.name, installerVersion, last_seen = lifetime[end] | sort installerVersion asc` — `from:-30d` matters, because a rejected agent stops reporting and drops out of the default 2 h window — see **FAQ-04: Managing OneAgent updates on Dynatrace SaaS**
Connection errors	Proxy required	Configure ActiveGate or proxy

Network Verification

```
# Test connectivity to Dynatrace
curl -v https://{tenant-id}.live.dynatrace.com/api/v1/time

# Check DNS resolution
nslookup {tenant-id}.live.dynatrace.com

# Test with OpenSSL
openssl s_client -connect {tenant-id}.live.dynatrace.com:443
```

Process Restart Requirement

OneAgent injects into running processes. After initial installation:

- **New processes** - Automatically monitored
- **Existing processes** - Restart required for full monitoring

For deep code-level visibility, restart:
- Application servers (Tomcat, JBoss, etc.)

- Web servers (Apache, Nginx, IIS)
- .NET/Java applications
- Node.js applications

8. Next Steps

With OneAgent deployed and verified:

1. **ONBRD-06: Organizing Your Environment** — Set up tags, segments, naming conventions, and `dt.security_context`
2. Wait 15-30 minutes for full topology discovery
3. Check Smartscape for service dependencies
4. Plan broader rollout based on pilot results

Where to Go Deeper

- **K8S series** (15 notebooks) — Cluster monitoring, DynaKube depth, GitOps for K8s deployments
- **FAQ-01** — Host group naming strategy (decide before first install)
- **FAQ-02** — Tagging sources, standards, and strategy (primary tags vs auto-tags vs cloud tags)

Deployment Checklist

- [ ] Pilot hosts deployed (2-5 hosts)
- [ ] Hosts appearing in Dynatrace
- [ ] No hosts in `OFFLINE` state over the last 7 days (state query in section 6)
- [ ] Services being discovered
- [ ] `oneagentctl --get-server` shows the expected endpoints
- [ ] Smartscape showing topology
- [ ] Host groups set at install time (not retrofitted later)
- [ ] Primary tags (environment, team, `dt.security_context`) emitted at source
- [ ] Windows hosts: Npcap install path planned (required from OneAgent 1.337)

Summary

In this notebook, you learned:

- What OneAgent does and how it works
- Phased deployment strategy
- How to generate deployment tokens
- Installation methods for Linux, Windows, and Kubernetes
- Setting primary tags and host groups at install time (OneAgent 1.333+)
- Windows Npcap requirement (OneAgent 1.337)
- How to verify successful deployment
- Common troubleshooting steps

References

- OneAgent Overview
- Linux Installation
- Windows Installation
- Kubernetes Operator
- Deployment API
- Deployment API - Download latest OneAgent (DT docs)
- OneAgent Attribute Enrichment
- What's new in Dynatrace SaaS 1.347 (DT docs) — *"Starting with this release, Dynatrace rejects connections from OneAgent versions 1.241 and earlier."*

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.